

Laporan Tugas Kecil 1

Strategi Algoritma Semester II 2026

- Bahasa Pemrograman yang digunakan : Python
- Penjelasan kode :

Terdapat sebuah fungsi utama bernama “helperChecker” yang bersifat rekursif untuk mengecek seluruh kemungkinan penaruhan queen sehingga menemui satu buah solusi yang memenuhi syarat. Terdapat fungsi “verify” yang akan mengembalikan true jika posisi queen tidak saling bersebelahan dengan queen lain, lalu posisi queen tidak satu kolom dan baris dengan queen lainnya. Fungsi “add” dan “addQueens” berturut-turut untuk menambah menambah satu pada satu variabel dan untuk menambahkan jumlah queen yang telah memenuhi syarat penempatan pada array “queens”. Fungsi “removeQueens” digunakan untuk menghapus queen dari array “queens”. Fungsi “makeSector” akan menghitung berapa banyak sektor yang terdapat di dalam papan.

Pertama program akan meminta masukan ukuran papan dan papannya sendiri dimana dapat dalam berbentuk input langsung, gambar, ataupun file .txt. Setelah itu, program akan masuk ke fungsi “helperChecker” untuk mengecek semua kemungkinan posisi dengan iterasi setiap kolom hingga akhir, lalu memulai ulang pada baris berikutnya sehingga menemukan posisi dimana queens mengikuti peraturan. Peraturan disini adalah posisi tidak bersebelahan dan tidak pada kolom/baris sama, serta tidak ada lebih dari satu queen pada satu sektor, dimana array sektor memiliki kolom boolean untuk mengatakan apakah sektor tersebut ditempatkan atau tidak. “addQueens” akan terpanggil setelah menemui posisi yang mengikuti peraturan, dimana array “queens” akan menyimpan koordinat x dan y, serta sektor queen tersebut berada. Rekursif akan berhenti jika variabel boolean “found” bernilai true, yaitu ketika ukuran array “queens” sama dengan jumlah queens yang ingin ditempatkan pada papan. Jika bernilai false, maka akan memasuki fungsi “helperChecker” sehingga semua queen ditempatkan. Jika ruang di papan untuk pengecekan sudah kosong dan semua queens belum ditempatkan, maka akan dilakukan backtracking. Fungsi “removeQueens” akan terpanggil untuk menghilangkan queen yang baru ditempatkan, sehingga posisi baru untuk queen tersebut ditemukan agar queen selanjutnya memiliki tempat untuk penempatan. Sebab, terdapat kemungkinan jika queen tidak dimajukan posisinya, maka posisi queen berikutnya tidak ada. Setelah semua queen sudah ditempatkan, program akan keluar dan menampilkan papan setelah penempatan semua queen, durasi pencarian, dan alternatif-alternatif yang dikunjungi. Serta, diberikan opsi untuk melakukan save papan.

- Pseudocode:

FUNCTION buildFinalBoard(board, queens, n)

```
CREATE newBoard[n][n]
```

```
FOR i ← 0 TO n-1 DO  
  FOR j ← 0 TO n-1 DO  
    IF (i, j) EXISTS IN queens THEN  
      newBoard[i][j] ← '#'  
    ELSE  
      newBoard[i][j] ← board[i][j]  
    END IF  
  END FOR  
END FOR
```

```
RETURN newBoard  
END FUNCTION
```

```
FUNCTION verify(board, queens, sectors, x, y)
```

```
FOR each queen IN queens DO  
  IF |queen.x - x| ≤ 1 AND |queen.y - y| ≤ 1 THEN  
    RETURN FALSE  
  END IF
```

```
  IF queen.x == x OR queen.y == y THEN  
    RETURN FALSE  
  END IF  
END FOR
```

```
FOR each sector IN sectors DO  
  IF board[x][y] == sector.letter AND sector.used == TRUE THEN  
    RETURN FALSE  
  END IF  
END FOR
```

```
RETURN TRUE  
END FUNCTION
```

```
FUNCTION addQueen(board, queens, sectors, alt, x, y)
```

```
alt ← alt + 1
```

```
letter ← board[x][y]
```

```
ADD (x, y, letter) TO queens
```

```
FOR each sector IN sectors DO  
  IF sector.letter == letter THEN  
    sector.used ← TRUE
```

```
        BREAK
    END IF
END FOR

END FUNCTION
```

```
FUNCTION removeQueen(queens, sectors)
```

```
    lastQueen ← REMOVE last element from queens
    letter ← lastQueen.letter
```

```
    FOR each sector IN sectors DO
        IF sector.letter == letter THEN
            sector.used ← FALSE
            BREAK
        END IF
    END FOR
```

```
END FUNCTION
```

```
PROCEDURE helperChecker(board, queens, sectors, found, alt, x, y, n)
```

```
    IF  $x \geq n$  THEN
        RETURN
    END IF
```

```
    WHILE  $x < n$  AND verify(board, queens, sectors, x, y) == FALSE DO
         $y \leftarrow y + 1$ 
        IF  $y \geq n$  THEN
             $y \leftarrow 0$ 
             $x \leftarrow x + 1$ 
        END IF
    END WHILE
```

```
    IF  $x \geq n$  THEN
        RETURN
    END IF
```

```
    CALL addQueen(board, queens, sectors, alt, x, y)
```

```
    IF size(queens) == n THEN
        found ← TRUE
    END IF
```

```
    IF found == FALSE THEN
        CALL helperChecker(board, queens, sectors, found, alt, x, y, n)
    ELSE
        RETURN
    END IF
```

```

IF found == FALSE THEN
    CALL removeQueen(queens, sectors)

     $y \leftarrow y + 1$ 
    IF  $y \geq n$  THEN
         $y \leftarrow 0$ 
         $x \leftarrow x + 1$ 
    END IF

    CALL helperChecker(board, queens, sectors, found, alt, x, y, n)
END IF

END PROCEDURE

PROGRAM QueensSolver

INPUT n
FOR  $i \leftarrow 0$  TO  $n-1$  DO
    INPUT row
    board[i]  $\leftarrow$  row
END FOR

queens  $\leftarrow$  empty list
sectors  $\leftarrow$  empty list
found  $\leftarrow$  FALSE
alt  $\leftarrow$  0

CALL makeSector(board, sectors)

start_time  $\leftarrow$  current_time()

CALL helperChecker(board, queens, sectors, found, alt, 0, 0, n)

end_time  $\leftarrow$  current_time()

finalBoard  $\leftarrow$  buildFinalBoard(board, queens, n)

PRINT finalBoard
PRINT alt
PRINT execution_time
END PROGRAM

```

- Test Case

No	Input	Output
1	<pre> 1 AAAAABBB 2 CCCCCDDB 3 EEEFFDDB 4 GGEFHDDDB 5 GGIIHDDDB 6 JJIIKLLL 7 JJMMKLLL 8 JJMMNNNN </pre>	<pre> # A A A B B B C C # C C D D B E E E F # D D B G # E F H D D B G G I I H # D B J J I I K L L # J J M # K L L L J J M M N N # N Jumlah Kasus yang ditinjau Duration time : 1.00 ms Apakah ingin save? (Y/n) </pre>
		<pre> 1 #AAAABBB 2 CC#CCDDB 3 EEEF#DDB 4 G#EFHDDDB 5 GGIIH#DB 6 JJIIKLL# 7 JJM#KLLL 8 JJMMNN#N 9 </pre>
2	<pre> Tucil1_13524041 > test > test2_txt(N=9).txt 1 AAABBCCCD 2 ABBBBCECD 3 ABBBDCECD 4 AAABDCCCD 5 BBBBDDDDD 6 FGGGDHDD 7 FGIGDDHDD 8 FGIGDDHDD 9 FGGGDHDDH </pre>	<pre> === LIVE UPDATE (Iterasi: 300000) === # . . . # # # . . . # # A A A B B C C # D A B B B # C E C D A B B B D C # C D A # A B D C C C D B B B B D # D D D F G G # D D H D D # G I G D D H D D F G # G D D H D D F G G D D H H # Jumlah Kasus yang ditinjau : 307100 Duration time : 3708.82 ms Apakah ingin save? (Y/n) </pre>

		<pre>1 AAABBCC#D 2 ABBB#CECD 3 ABBBDC#CD 4 A#ABDCCCD 5 BBBBD#DDD 6 FGG#DDHDD 7 #GIGDDHDD 8 FG#GDDHDD 9 FGGGDDHH# 10 </pre>
3	<pre>root_13524041 / test / 1 AAABB 2 AACBB 3 CCCC D 4 EEEDD 5 EEEDD</pre>	<pre># A A B B A A C # B C # C C D E E E D # E E # D D Jumlah Kasus yang ditinjau : 19 Duration time : 0.00 ms Apakah ingin save? (Y/n) </pre>
		<pre>1 #AABB 2 AAC#B 3 C#CCD 4 EEED# 5 EE#DD 6 </pre>
4		<pre># A A B B B C C A A A A # B C C A # D B B B C C A D D C C C # C E F F # C C C C E F E F G # H H E E F F G H H # E E # G G G G G Jumlah Kasus yang ditinjau : 145 Duration time : 3.01 ms Apakah ingin save? (Y/n) </pre>

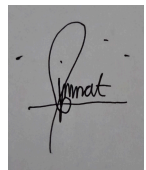
5		<pre># A A A B B A A A A # B A C # C B B C C C C D # E # E E D D E E F # F D Jumlah Kasus yang ditinjau : 22 Duration time : 0.00 ms Apakah ingin save? (Y/n)</pre>

- Link Github : https://github.com/N8NAS/Tucil1_13524041.git
- sdf

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	x	
2	Program berhasil di jalankan	x	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	x	

4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	x	
5	Program memiliki Graphical User Interface (GUI)		x
6	Program dapat menyimpan solusi dalam bentuk file gambar	x	

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Nathan Adhika Santosa